Análise Experimental de Desempenho em Arranjos Ordenados

Engenharia de Software e Estruturas de Dados Avançadas

Avaliando o impacto da ordenação no desempenho de estruturas de dados em Java.

Gabriel Retzlaff

Contexto

Implementação de uma estrutura de dados de Arranjo Ordenado customizada em Java.

Objetivo

Quantificar como a direção da ordenação afeta as operações de inserção e exclusão.

Escopo

Uso de Busca Binária e deslocamento de memória (shift) para manutenção da ordem.

Importância

Compreender o custo real de manutenção da ordem em sistemas de alta performance.

Arquitetura da Estrutura (UML)

Design polimórfico permite flexibilidade na lógica de comparação

ArranjoOrdenado

```
- elementos: int[]
- tamanho: int
- crescente: boolean

+ inserir(int): void
+ excluir(int): void
+ buscar(int): int
+ encontrarPosicao(int): int
```

Campos Principais

O container físico dos dados é um array de inteiros, com controle de tamanho atual e flag de direção.

Métodos Core

Operações fundamentais para manipulação e consulta, otimizadas para manter a ordem dos elementos.

Diferencial Polimórfico

A lógica de comparação é desacoplada, permitindo inverter a ordem (Crescente/Decrescente) sem alterar a estrutura base.

Implementação Técnica e Código Chave

Otimização de busca com O(log n) e manutenção de ordem com O(n)

Localização Eficiente

A Busca Binária reduz o tempo de procura para **O(log n)**, minimizando comparações em grandes volumes de dados.

Manutenção da Ordem

O deslocamento (Shift) de elementos garante a integridade da ordem, com custo computacional de **O(n)**.

Performance JVM

O uso de `System.arraycopy` é preferível a laços manuais por ser uma operação nativa otimizada da JVM.

// Exemplo de Manutenção de Ordem em Java

```java
// Deslocamento eficiente de memória
public void excluir(int index) {
    int numMovidos = tamanho - index - 1;
    if (numMovidos > 0) {
        System.arraycopy(elementos, index + 1,
                         elementos, index,
                         numMovidos);
    }
    elementos[--tamanho] = 0;
}
```

Metodologia do Experimento Científico

Rigor estatístico com 100 execuções independentes e 100.000 elementos

Cenário de Teste

Processamento de **100.000 elementos inteiros** para garantir uma carga de trabalho significativa e mensurável.

Amostragem

Realização de **100 execuções independentes** para cada cenário, eliminando ruídos de processamento e variações da JVM.

Instrumentação

Uso de **System.nanoTime()** para capturar a performance com precisão de nanossegundos, essencial para microbenchmarks.

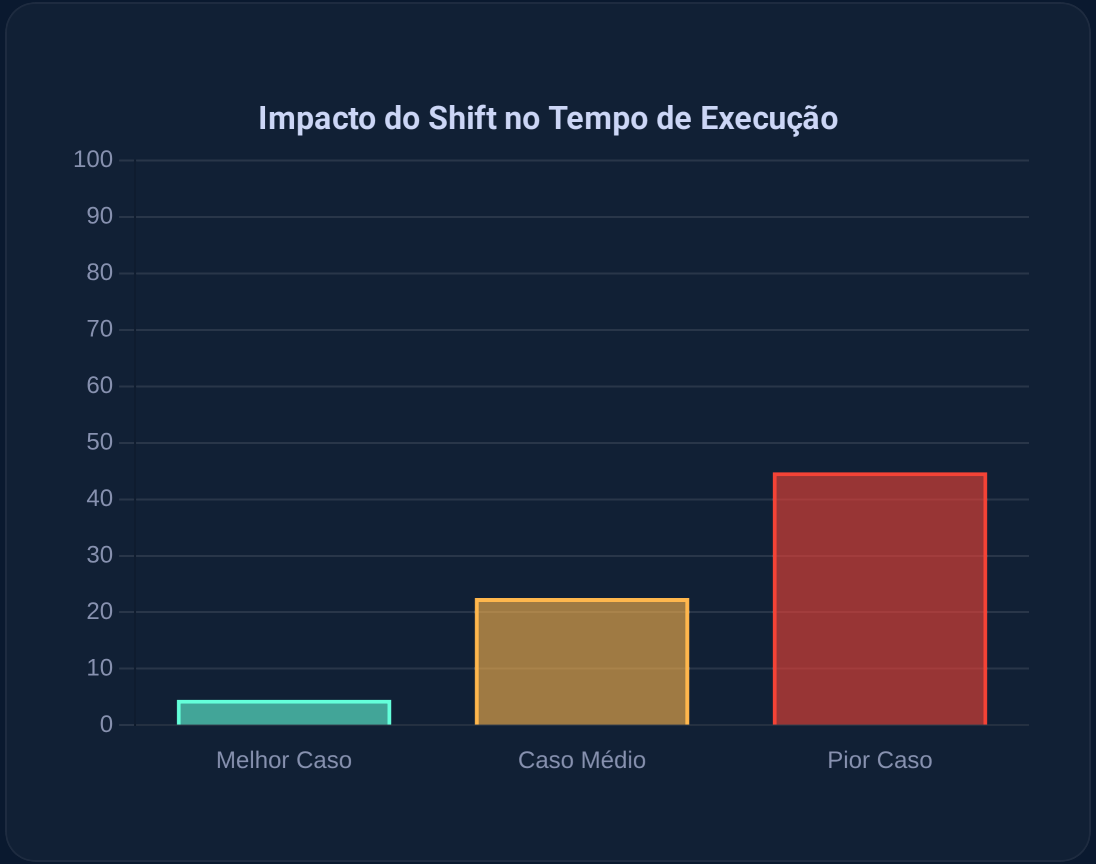
Variáveis de Entrada

Testes realizados com entradas em ordem **Crescente, Decrescente e Aleatória** para mapear diferentes comportamentos.

Resultados de Inserção (Dados Experimentais)

O esforço de deslocamento de memória define o tempo de inserção

Cenário de Entrada	Estrutura Alvo	Esforço de Shift	Desempenho
Crescente	Crescente	Mínimo	**Melhor Caso**
Decrescente	Crescente	Máximo	**Pior Caso**
Aleatória	Crescente	Médio	**Caso Médio**



** Métrica: Os resultados são expressos através da Média Aritmética e do Desvio Padrão (σ) para validar a consistência dos dados.*

Resultados de Exclusão e Análise de Tempo

Performance de remoção total em larga escala

Análise de Exclusão

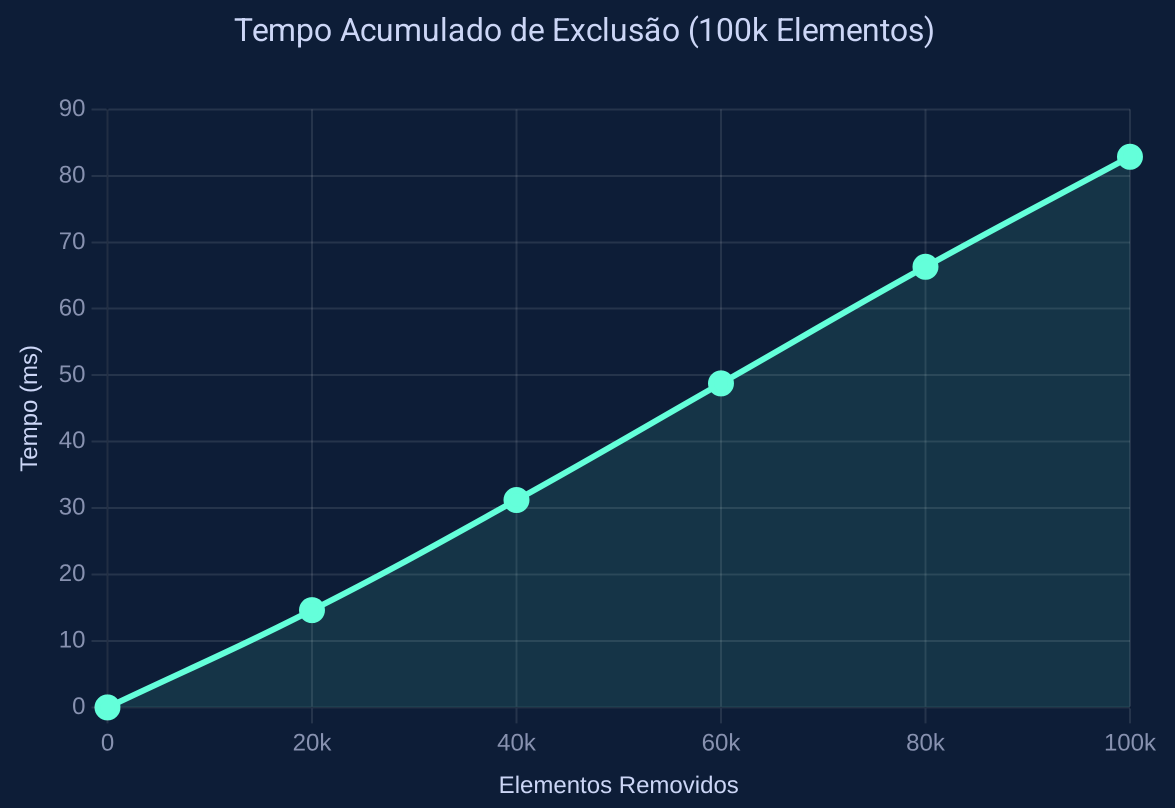
Medição do tempo total necessário para remover todos os 100.000 elementos da estrutura de forma sequencial.

Comportamento de Shift

Assim como na inserção, a exclusão exige o rearranjo dos elementos subsequentes para preencher as lacunas na memória.

Impacto da Localidade

A performance é influenciada pela forma como a JVM gerencia o cache e a localidade dos dados durante os shifts sucessivos.



Validação Estatística: Teste de Wilcoxon

Diferenças de performance são estatisticamente significantes ($p < 0.05$)

Objetivo do Teste

Validar se as variações de tempo observadas entre os cenários de ordenação são reais ou fruto de ruído experimental.

Método Aplicado

Utilização do **Teste de Wilcoxon Signed-Rank**, ideal para comparar amostras pareadas sem assumir distribuição normal.

Conclusão Científica

A hipótese nula foi rejeitada, provando que a direção da ordenação causa um impacto sistemático e previsível no desempenho.

VALOR DE SIGNIFICÂNCIA

$$p < 0.05$$

Resultado Significante

** Teste realizado sobre 100 amostras independentes de 100.000 elementos cada.*

Análise de Complexidade Big O

Eficiência em buscas vs. Custo em modificações

Operação	Complexidade
Busca (Binária)	`O(log n)`
Inserção	`O(n)`
Exclusão	`O(n)`

Vantagem de Busca

A estrutura é extremamente eficiente para leitura e localização de dados, superando arranjos não ordenados.

Gargalo de Escrita

Ineficiente para grandes volumes de escrita devido ao custo de deslocamento de memória (shift) necessário para manter a ordem.

VERDITO: Ideal para sistemas "Read-Heavy". Evite em cenários de alta frequência de escrita.

Conclusões e Recomendações Técnicas

Escolhendo a estrutura certa para o problema certo

Impacto da Ordenação

Manter a ordem aumenta muito o tempo por causa dos shifts de memória.

Recomendação de Estruturas

Para muitas escritas e buscas, prefira árvores balanceadas (AVL/Red-Black).

Limites da Otimização

System.arraycopy melhora prática, mas não muda a complexidade O(n).

Próximos Passos

Estender para tipos genéricos e medir memória/cache.

Obrigado pela atenção! Dúvidas e discussões técnicas são bem-vindas.